Design and Verification of a 16-bit Arithmetic Logic Unit (ALU) using Verilog HDL

Processor Design – Project Report

Tool: Xilinx/AMD Vivado Design Suite (Behavioral Simulation)

Student name	Rizwan Ahmad Hakak
	Taha Asif
Enroll number	2026MECEME001
	2026MECEME002
Programme	M.Tech, Electronics & Communication Engineering, NIT Srinagar
Course	Advance Computer Architecture
Submission date	11 November 2026
GitHub repository	

1. Abstract

The Arithmetic Logic Unit (ALU) is the part of a processor that performs arithmetic and logical operations on data. This project presents the design of a 16-bit combinational ALU in Verilog HDL. The ALU accepts two 16-bit operands and a 4-bit opcode, supports 16 operations (arithmetic, logical, shift/rotate and compare) and produces a 16-bit result together with four status flags: carry, overflow, zero and negative. A self-checking testbench applies 21 directed test vectors and compares every result and flag against hand-computed expected values. The design is simulated in Vivado, and results are reported through the simulator console and waveform.

2. Introduction and Objectives

Every processor datapath contains an ALU. The control unit places an operation code on the ALU's control lines, the ALU computes the result from the operands, and the flags it raises (zero, carry, overflow, negative) are used by conditional branch instructions. The objectives of this project are:

- Design a clear, readable 16-bit ALU in synthesizable Verilog.
- Support arithmetic, logic, shift/rotate and comparison operations selected by a 4-bit opcode.
- Generate the standard status flags used by a processor.
- Verify the design with a self-checking testbench and benchmark (expected) values.
- Document the design, including block diagram, applications and a short survey.

3. Block Diagram of the ALU System

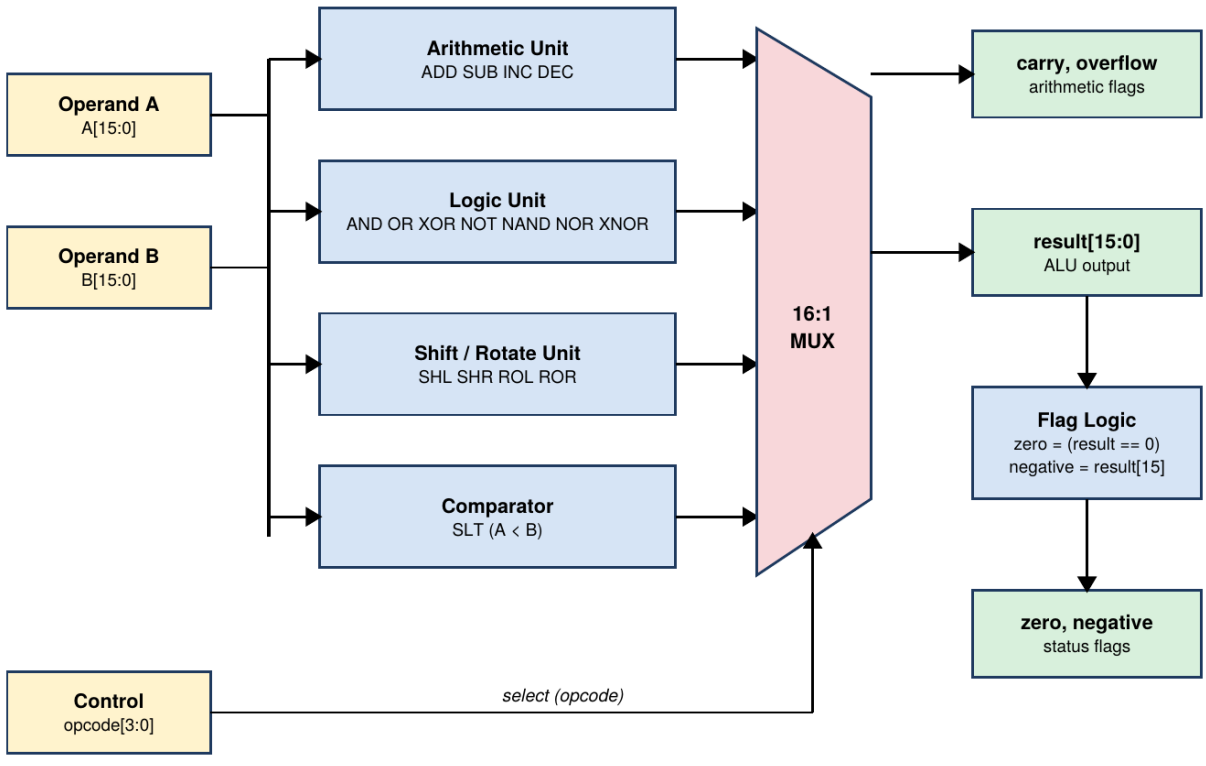


Figure 1: Block diagram of the 16-bit ALU system

Explanation of each block

Block	Purpose
Operand A, B (16-bit)	Input data on which the operation is performed. Treated as unsigned or as 2's complement signed numbers depending on the flag being interpreted.
Control: opcode (4-bit)	Selects which of the 16 operations appears at the output. 4 bits give $2^4 = 16$ combinations.
Arithmetic Unit	Performs ADD, SUB, INC and DEC using a 17-bit result so that the extra top bit becomes the carry (or borrow) output. Also produces the signed overflow flag for ADD and SUB.
Logic Unit	Performs bitwise AND, OR, XOR, NOT, NAND, NOR and XNOR. Each result bit depends only on the corresponding bits of A and B.
Shift / Rotate Unit	SHL and SHR shift A by one position and send the bit that falls out to carry. ROL and ROR rotate A so the bit leaving one end re-enters at the other end.
Comparator	SLT outputs 1 if $A < B$ (unsigned), otherwise 0. Used to implement comparison instructions.
16:1 Multiplexer	Implemented by the `case` statement. Passes only the output of the unit chosen by the opcode to the result and flag outputs.
Flag Logic	`zero` = 1 when all result bits are 0 (NOR of 16 bits). `negative` = `result[15]`, the sign bit in 2's complement.

4. Operation Table (Instruction Set of the ALU)

Opcode	Name	Category	Operation
0000	ADD	Arithmetic	$A + B$
0001	SUB	Arithmetic	$A - B$ (carry = borrow)
0010	AND	Logic	A AND B
0011	OR	Logic	A OR B
0100	XOR	Logic	A XOR B
0101	NOT	Logic	NOT A
0110	SHL	Shift	A shifted left by 1 (carry = old MSB)
0111	SHR	Shift	A shifted right by 1 (carry = old LSB)
1000	INC	Arithmetic	$A + 1$
1001	DEC	Arithmetic	$A - 1$ (carry = borrow)
1010	NAND	Logic	NOT (A AND B)
1011	NOR	Logic	NOT (A OR B)
1100	XNOR	Logic	NOT (A XOR B)
1101	SLT	Compare	1 if $A < B$, else 0
1110	ROL	Rotate	Rotate A left by 1
1111	ROR	Rotate	Rotate A right by 1

Flag definitions

- **carry**: carry out of bit 15 for addition and increment; borrow (1 when $A < B$) for subtraction and decrement; the bit shifted out for SHL/SHR. It is 0 for logic, compare and rotate operations.
- **overflow**: signed overflow. For ADD it is set when A and B have the same sign but the result has a different sign. For SUB it is set when A and B have different signs and the result's sign differs from A's.
- **zero**: set when the 16-bit result is all zeros.
- **negative**: copy of the result MSB.

5. Important Pieces of Code and Their Purpose

The complete commented source is in `alu_16bit.v` and `alu_16bit_tb.v` (also listed in the Appendix). The key parts are explained below.

5.1 Module declaration and ports

```
module alu_16bit (
    input  wire [15:0] A, B,
    input  wire [3:0]  opcode,
    output reg  [15:0] result,
    output reg         carry, overflow,
    output wire        zero, negative
);
```

The module is the hardware block. Inputs are wires. Outputs assigned inside an `always` block must be declared `reg`; here "reg" only means a procedural variable, and the synthesized hardware is purely combinational logic with no flip-flops. `zero` and `negative` use `assign`, so they are wires.

5.2 Combinational always block and default values

```
always @(*) begin
    result = 16'b0; carry = 1'b0; overflow = 1'b0;
    case (opcode)
        ADD: begin
            {carry, result} = A + B;
            overflow = (A[15]==B[15]) && (result[15]!=A[15]);
        end
        ...
    endcase
end
```

`@(*)` makes the block re-evaluate whenever any input changes, which describes combinational logic. Default values are assigned first so every output has a value on every path; otherwise synthesis would infer unwanted latches. The concatenation `{carry, result}` is a 17-bit target, so the addition's carry-out is captured automatically.

5.3 Shift, rotate and flags

```
SHL:  {carry, result} = {A, 1'b0};
SHR:  {result, carry} = {1'b0, A};
ROL:  result = {A[14:0], A[15]};
ROR:  result = {A[0], A[15:1]};
```

```
...
assign zero     = (result == 16'b0);
assign negative = result[15];
```

Concatenation builds shifted and rotated values without any shift operator. For rotation the bit that leaves one end is re-attached at the other end, so no information is lost.

5.4 Self-checking testbench

```
task check;
    input [3:0] op; input [15:0] a_in, b_in, exp_result;
    input exp_carry, exp_overflow;
    begin
        A = a_in; B = b_in; opcode = op;
        #10;
        if (result===exp_result && carry===exp_carry
            && overflow===exp_overflow) pass_count = pass_count+1;
        else fail_count = fail_count+1;
    end
endtask
```

The testbench has no ports. Its inputs are `reg` because the testbench drives them, and the ALU outputs are `wire` because it only observes them. The `task` applies one test vector, waits 10 ns for the logic to settle and compares with the expected values using `===` (which also catches x/z values). A final `$display` prints the total pass and fail counts.

6. Testing and Verification

The testbench applies 21 directed vectors covering every opcode, including corner cases: carry out (FFFF + 1), signed overflow (7FFF + 1 and 8000 − 1), borrow (4 − 9), zero result, and both outcomes of the compare operation. The benchmark values in the table are computed by hand and cross-checked with an independent software model. All values are hexadecimal except the flags.

#	Op	A	B	Expected result	C	V	Z	N
1	ADD	0005	0003	0008	0	0	0	0
2	ADD	FFFF	0001	0000	1	0	1	0
3	ADD	7FFF	0001	8000	0	1	0	1
4	SUB	0009	0004	0005	0	0	0	0
5	SUB	0004	0009	FFFB	1	0	0	1
6	SUB	8000	0001	7FFF	0	1	0	0
7	AND	F0F0	FF00	F000	0	0	0	1
8	OR	F0F0	0F0F	FFFF	0	0	0	1
9	XOR	AAAA	5555	FFFF	0	0	0	1
10	NOT	00FF	0000	FF00	0	0	0	1
11	SHL	8001	0000	0002	1	0	0	0
12	SHR	0003	0000	0001	1	0	0	0
13	INC	FFFF	0000	0000	1	0	1	0
14	DEC	0000	0000	FFFF	1	0	0	1
15	NAND	FFFF	FFFF	0000	0	0	1	0
16	NOR	0000	0000	FFFF	0	0	0	1
17	XNOR	AAAA	AAAA	FFFF	0	0	0	1
18	SLT	0003	0005	0001	0	0	0	0
19	SLT	0005	0003	0000	0	0	1	0
20	ROL	8001	0000	0003	0	0	0	0
21	ROR	8001	0000	C000	0	0	0	1

Expected console output

When the simulation is run in Vivado (Run Simulation, then Run Behavioral Simulation), the Tcl console lists one PASS line per test and finishes with the summary below.

```
PASS | op=0000 A=0005 B=0003 | result=0008 C=0 V=0 Z=0 N=0
PASS | op=0000 A=ffff B=0001 | result=0000 C=1 V=0 Z=1 N=0
PASS | op=0000 A=7fff B=0001 | result=8000 C=0 V=1 Z=0 N=1
PASS | op=0001 A=0009 B=0004 | result=0005 C=0 V=0 Z=0 N=0
...
----------------------------------------
TOTAL PASS = 21, FAIL = 0
----------------------------------------
```

Simulation evidence

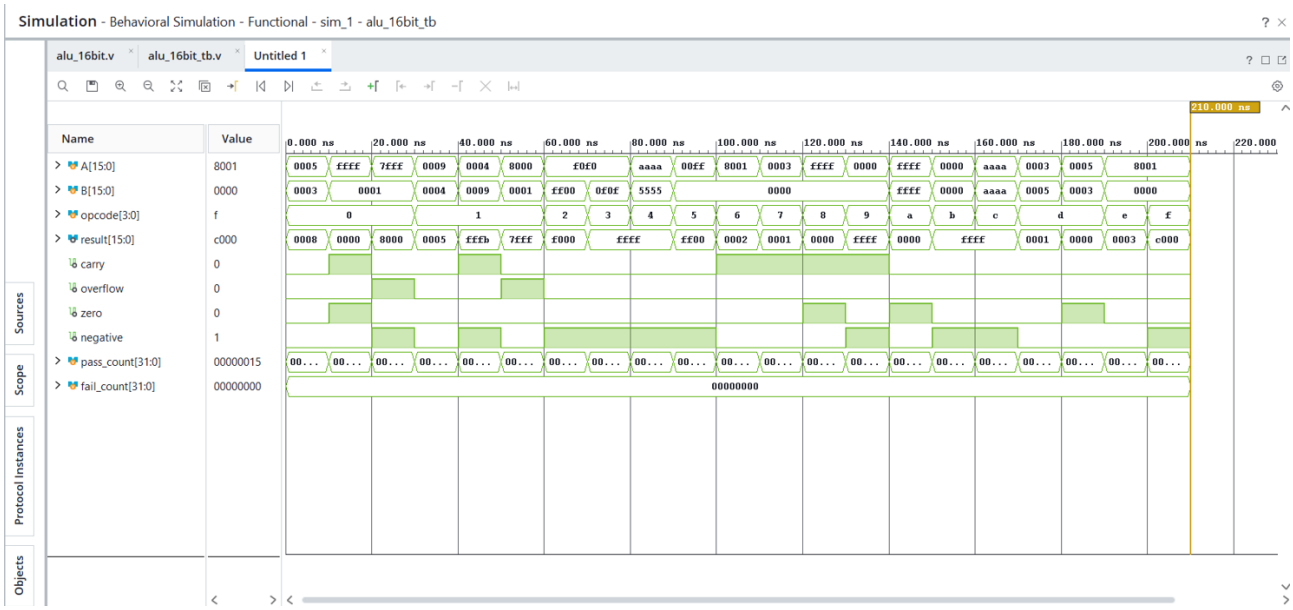


Figure 2: Behavioural simulation waveform

Waveform analysis. Figure 2 spans 0–210 ns. The testbench changes the inputs every 10 ns, so `opcode` steps from 0 to F while `A`, `B` and `result` update with it, which shows the ALU is purely combinational. The flags behave as expected: `zero` goes high for the tests whose result is 0000 (FFFF + 1, INC of FFFF, NAND and SLT false), `negative` follows the MSB of the result, and `carry` rises for the carry, borrow and shift-out cases. At the end, `pass_count` reads 15 in hex (21 in decimal) and `fail_count` stays at 0.

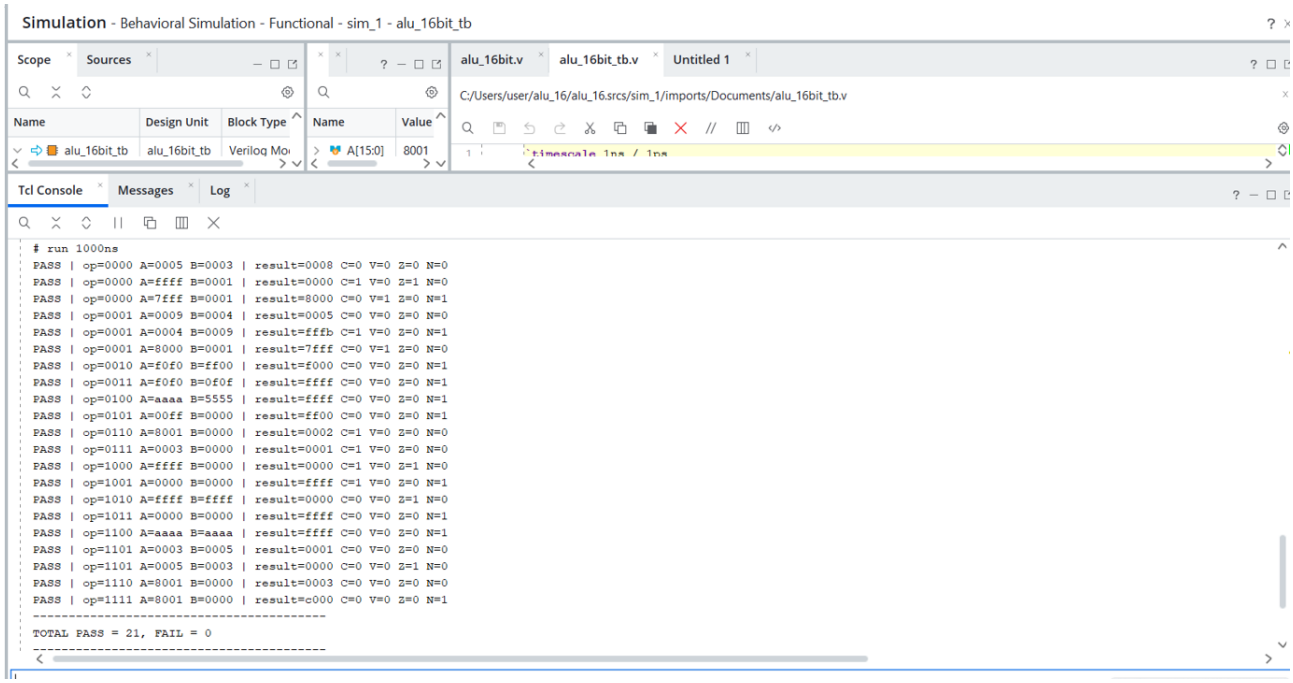


Figure 3: Tcl console output showing `TOTAL PASS = 21, FAIL = 0`

Console analysis. Each of the 21 test vectors in Figure 3 prints one PASS line showing the opcode, both operands, the result and the four flags. The testbench compares these against the expected values using `===`, and the final summary reports `TOTAL PASS = 21, FAIL = 0`, so every operation and flag

matched its benchmark value.

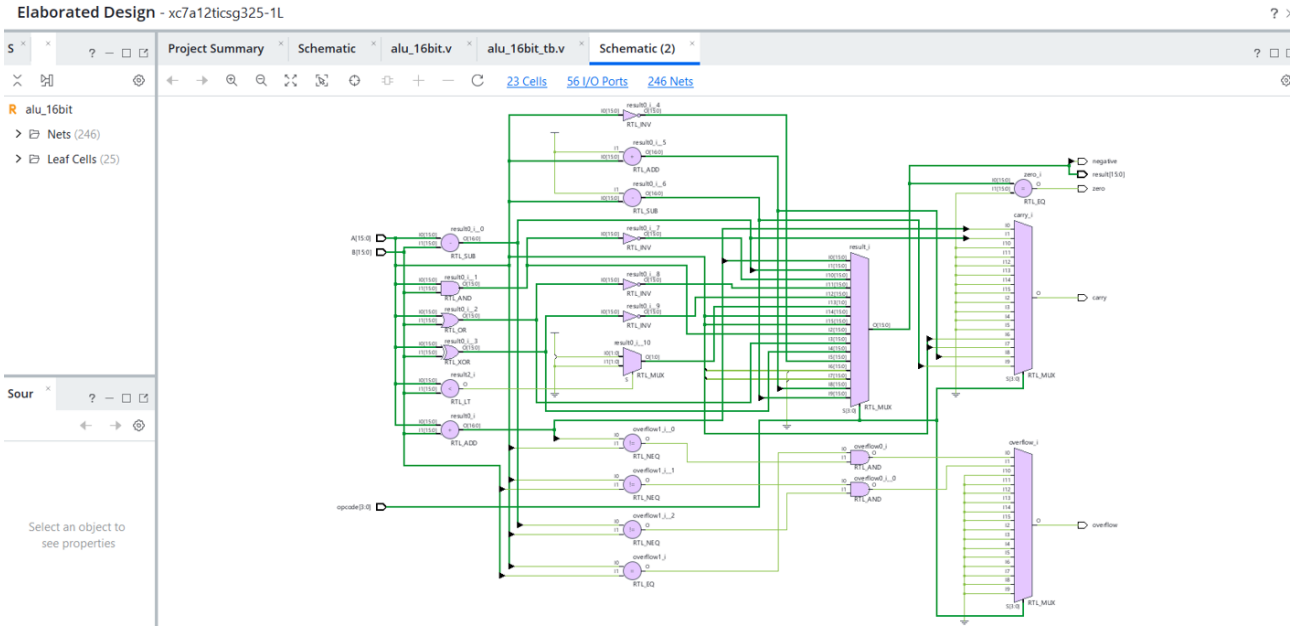


Figure 4: RTL schematic of the 16-bit ALU

Schematic analysis. The elaborated design in Figure 4 contains 23 cells, 56 I/O ports (36 inputs: `A`, `B`, `opcode`; 20 outputs: `result` and the four flags) and 246 nets. Vivado has mapped the Verilog operators to RTL adder, subtractor, AND/OR/XOR/inverter and comparator blocks, and the `case` statement to multiplexers that select the `result`, `carry` and `overflow` outputs from the `opcode`. This matches the block diagram in Figure 1.

Summary. The waveform, console output and schematic together confirm that the 16-bit ALU is functionally correct for all 16 operations and its flag outputs.

7. Applications of the ALU System

- **General-purpose CPUs and microprocessors:** the execution stage of the datapath, where arithmetic, logic and compare instructions are carried out.
- **Microcontrollers and embedded systems:** control, sensor-data processing and address calculation.
- **Digital signal processing:** add, subtract and shift operations for filters and scaling.
- **FPGA soft-core processors:** this design can be reused inside a custom 16-bit RISC core.
- **Address generation and loop counters:** INC/DEC and compare operations are used to step through memory and test loop conditions.
- **Education and prototyping:** a base for building a full datapath with registers, a control unit and memory.

8. Short Survey: Comparison with Existing Approaches

ALUs can be built in several ways, and the right choice depends on the speed, area and flexibility required. The table compares the present design with common alternatives described in standard digital-design textbooks.

Approach	Description	Advantages	Limitations
This design (behavioral, operator-based)	Uses Verilog operators (+, −, &, |) inside one case statement; synthesis tool chooses the adder structure.	Short, readable, easy to extend; 16 operations and 4 flags; fully combinational.	Speed depends on the tool's adder; not pipelined; no multiply/divide.
Structural ripple-carry ALU	Built from 1-bit full adders chained together.	Smallest area; simple to understand.	Carry must travel through all 16 stages, so delay grows linearly with width.
Carry-lookahead ALU	Computes carries in parallel using generate and propagate signals.	Much lower adder delay.	More gates and wiring; harder to write and verify by hand.
74181-style bit-slice ALU	Classic 4-bit ALU chip with function-select lines; slices are cascaded for wider words.	Historically important; many functions per slice.	Obsolete as a discrete part; cascading adds complexity.
Pipelined / registered ALU	Registers placed on inputs/outputs or inside the datapath.	Higher clock frequency in a synchronous processor.	Adds latency and registers; needs a clock and control.
FPGA DSP-slice ALU	Uses the dedicated arithmetic blocks inside the FPGA.	Very high speed and low logic usage on that device.	Device-specific and less portable.

The present design deliberately favours clarity and completeness of the instruction set. Because the adder is described with the `+` operator, Vivado synthesis maps it to efficient FPGA carry logic, which gives good performance without hand-written lookahead logic. A natural improvement is to add a clocked output register so the ALU can be used in a pipelined processor.

9. Conclusion and Future Work

A 16-bit ALU with 16 operations and four status flags was designed in Verilog HDL and verified with a self-checking testbench of 21 directed test vectors. The design is simple, synthesizable and easy to extend. Future work includes adding multiplication and signed comparison, adding a clocked register stage, and integrating the ALU into a complete 16-bit processor with a register file and control unit.

10. Project Files (GitHub Repository Contents)

File	Description
`alu_16bit.v`	Main design module
`alu_16bit_tb.v`	Self-checking testbench
`ALU_16bit_Project_Report.pdf`	This report
`screenshots/`	Vivado waveform and Tcl console screenshots
`README.md`	Project overview and instructions to run the simulation

11. References

[1] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, Pearson.

[2] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Prentice Hall.

[3] IEEE Std 1364-2005, *IEEE Standard for Verilog Hardware Description Language*.

[4] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, Morgan Kaufmann.

[5] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design*, Morgan Kaufmann.

[6] AMD (Xilinx), *Vivado Design Suite User Guide: Logic Simulation* (UG900).

[7] AMD (Xilinx), *Vivado Design Suite User Guide: Synthesis* (UG901).

A. Complete Source Code

A.1 alu_16bit.v

```
`timescale 1ns / 1ps

// 16-bit Arithmetic Logic Unit (combinational)
module alu_16bit (
    input  wire [15:0] A,        // first operand
    input  wire [15:0] B,        // second operand
    input  wire [3:0]  opcode,   // selects the operation (16 possible)
    output reg  [15:0] result,   // result of the operation
    output reg         carry,    // carry out (add) / borrow (sub) / shifted-out bit
    output reg         overflow, // signed overflow flag
    output wire        zero,     // 1 when result == 0
    output wire        negative  // 1 when result MSB is 1 (negative in 2's complement)
);

    // Opcode names make the code readable
    localparam ADD  = 4'b0000;
    localparam SUB  = 4'b0001;
    localparam AND_ = 4'b0010;
    localparam OR_  = 4'b0011;
    localparam XOR_ = 4'b0100;
    localparam NOT_ = 4'b0101;
    localparam SHL  = 4'b0110;
    localparam SHR  = 4'b0111;
    localparam INC  = 4'b1000;
    localparam DEC  = 4'b1001;
    localparam NAND_= 4'b1010;
    localparam NOR_ = 4'b1011;
    localparam XNOR_= 4'b1100;
    localparam SLT  = 4'b1101;   // set if A < B (unsigned)
    localparam ROL  = 4'b1110;
    localparam ROR  = 4'b1111;

    always @(*) begin
        // Default values: avoids unwanted latches
        result   = 16'b0;
        carry    = 1'b0;
        overflow = 1'b0;

        case (opcode)
            ADD: begin
                {carry, result} = A + B;
                overflow = (A[15] == B[15]) && (result[15] != A[15]);
            end
```

```
            SUB: begin
                {carry, result} = A - B;   // carry = 1 means borrow (A < B)
                overflow = (A[15] != B[15]) && (result[15] != A[15]);
            end
            AND_:  result = A & B;
            OR_:   result = A | B;
            XOR_:  result = A ^ B;
            NOT_:  result = ~A;
            SHL:   {carry, result} = {A, 1'b0};   // carry gets old MSB
            SHR:   {result, carry} = {1'b0, A};   // carry gets old LSB
            INC:   {carry, result} = A + 16'd1;
            DEC:   {carry, result} = A - 16'd1;
            NAND_: result = ~(A & B);
            NOR_:  result = ~(A | B);
            XNOR_: result = ~(A ^ B);
            SLT:   result = (A < B) ? 16'd1 : 16'd0;
            ROL:   result = {A[14:0], A[15]};
            ROR:   result = {A[0], A[15:1]};
            default: result = 16'b0;
        endcase
    end

    // Status flags derived continuously from result
    assign zero     = (result == 16'b0);
    assign negative = result[15];

endmodule
```

A.2 alu_16bit_tb.v

```
`timescale 1ns / 1ps

module alu_16bit_tb;

    // Testbench drives inputs -> reg ; observes outputs -> wire
    reg  [15:0] A, B;
    reg  [3:0]  opcode;
    wire [15:0] result;
    wire        carry, overflow, zero, negative;

    integer pass_count, fail_count;

    // Device Under Test
    alu_16bit uut (
        .A(A), .B(B), .opcode(opcode),
        .result(result), .carry(carry), .overflow(overflow),
        .zero(zero), .negative(negative)
    );

    // Reusable check routine: apply inputs, wait, compare with expected
    task check;
        input [3:0]  op;
        input [15:0] a_in, b_in;
        input [15:0] exp_result;
        input        exp_carry;
        input        exp_overflow;
        begin
            A = a_in; B = b_in; opcode = op;
            #10;  // let combinational logic settle
            if (result === exp_result && carry === exp_carry && overflow === exp_overflow) begin
                pass_count = pass_count + 1;
                $display("PASS | op=%b A=%h B=%h | result=%h C=%b V=%b Z=%b N=%b",
                         op, a_in, b_in, result, carry, overflow, zero, negative);
            end else begin
                fail_count = fail_count + 1;
                $display("FAIL | op=%b A=%h B=%h | got %h C=%b V=%b | expected %h C=%b V=%b",
                         op, a_in, b_in, result, carry, overflow,
                         exp_result, exp_carry, exp_overflow);
            end
```

```
        end
    endtask

    initial begin
        pass_count = 0;
        fail_count = 0;

        //     op     A       B       result  C  V
        check(4'b0000, 16'h0005, 16'h0003, 16'h0008, 0, 0); // ADD
        check(4'b0000, 16'hFFFF, 16'h0001, 16'h0000, 1, 0); // ADD carry, zero
        check(4'b0000, 16'h7FFF, 16'h0001, 16'h8000, 0, 1); // ADD signed overflow
        check(4'b0001, 16'h0009, 16'h0004, 16'h0005, 0, 0); // SUB
        check(4'b0001, 16'h0004, 16'h0009, 16'hFFFB, 1, 0); // SUB borrow
        check(4'b0001, 16'h8000, 16'h0001, 16'h7FFF, 0, 1); // SUB overflow
        check(4'b0010, 16'hF0F0, 16'hFF00, 16'hF000, 0, 0); // AND
        check(4'b0011, 16'hF0F0, 16'h0F0F, 16'hFFFF, 0, 0); // OR
        check(4'b0100, 16'hAAAA, 16'h5555, 16'hFFFF, 0, 0); // XOR
        check(4'b0101, 16'h00FF, 16'h0000, 16'hFF00, 0, 0); // NOT
        check(4'b0110, 16'h8001, 16'h0000, 16'h0002, 1, 0); // SHL
        check(4'b0111, 16'h0003, 16'h0000, 16'h0001, 1, 0); // SHR
        check(4'b1000, 16'hFFFF, 16'h0000, 16'h0000, 1, 0); // INC
        check(4'b1001, 16'h0000, 16'h0000, 16'hFFFF, 1, 0); // DEC
        check(4'b1010, 16'hFFFF, 16'hFFFF, 16'h0000, 0, 0); // NAND
        check(4'b1011, 16'h0000, 16'h0000, 16'hFFFF, 0, 0); // NOR
        check(4'b1100, 16'hAAAA, 16'hAAAA, 16'hFFFF, 0, 0); // XNOR
        check(4'b1101, 16'h0003, 16'h0005, 16'h0001, 0, 0); // SLT true
        check(4'b1101, 16'h0005, 16'h0003, 16'h0000, 0, 0); // SLT false
        check(4'b1110, 16'h8001, 16'h0000, 16'h0003, 0, 0); // ROL
        check(4'b1111, 16'h8001, 16'h0000, 16'hC000, 0, 0); // ROR

        $display("----------------------------------------");
        $display("TOTAL PASS = %0d, FAIL = %0d", pass_count, fail_count);
        $display("----------------------------------------");
        $finish;
    end

endmodule
```